

Key-Value Store v4

Heartbeat, failover e split brain

Architetture dei Sistemi Distribuiti

Lezione 13

Dopo replica e secondario, la domanda e':

- come capisco se il leader e' ancora vivo?
- quando un follower puo' promuoversi?
- cosa impedisce a due nodi di scrivere insieme?

Meccanismo minimale:

- il primary invia heartbeat periodici;
- il secondary aggiorna il proprio orologio locale;
- se il tempo supera una soglia, scatta il sospetto di guasto.

Timeout non e' verita'

Un timeout significa:

- non ho visto heartbeat abbastanza in fretta.

Non significa automaticamente:

- il leader e' certamente crashato.

```
if role == "secondary" and now - last_heartbeat > timeout:  
    term += 1  
    role = "primary"  
    leader_id = self_id
```

Qui nasce il failover.

Il timeout aiuta la liveness:

- evita di attendere per sempre un leader assente;
- permette al follower di tornare scrivibile.

Lo stesso timeout mette a rischio la safety:

- il leader puo' essere solo lento o isolato;
- il follower puo' promuoversi troppo presto;
- due nodi possono accettare scritture divergenti.

Caso tipico:

- ➊ A smette di inviare heartbeat;
- ➋ B si promuove;
- ➌ A resta convinto di essere leader;
- ➍ A e B accettano update diversi.

STATUS

OK node=A role=primary term=1 leader=A

Il laboratorio rende espliciti:

- ruolo percepito;
- termine;
- leader percepito.

Failover pulito:

- ① primary attivo;
- ② crash del primary;
- ③ attesa del timeout;
- ④ promozione del follower.

Split brain:

- 1 mettere in pausa gli heartbeat;
- 2 attendere la promozione del follower;
- 3 eseguire scritture su entrambi i nodi.

Perche' due nodi non bastano

Con due nodi manca una maggioranza indipendente.

- ogni nodo ha solo la propria vista locale;
- in caso di ambiguita', nessuno puo' arbitrare in modo robusto.

Heartbeat e timeout migliorano il failover.

- ma non bastano a garantire unicità del leader;
- serve una decisione distribuita più forte del solo sospetto locale.